NICLAHE Code Documentation and Explanation Report

(based on https://doi.org/10.18240/ijo.2025.12.02 "An improved neighbourhood-based contrast limited adaptive histogram equalization method for contrast enhancement on retinal images")

1. Overview of the Code

The code implements **NICLAHE** (Neighborhood-based Improved Contrast Limited Adaptive Histogram Equalization).

Unlike standard CLAHE, which uses the **same clip limit for every tile**, NICLAHE adjusts the clip limit **tile by tile** based on the local image content. Specifically, it looks at the **contrast of neighboring tiles** to decide how much enhancement a region should receive. This makes it especially suitable for medical images like retinal photographs, where we want to boost faint blood vessels without amplifying noise in flat areas.

The code is split into these logical blocks:

- Image loading and preparation
- Adaptive clip limit calculation (the "smart" part)
- Histogram equalization per tile with clipping
- Smooth blending of the enhanced tiles (bilinear interpolation)
- Saving the result

2. Image Loading and Preparation

python

```
img = Image.open(input_path).convert("L")
img_np = np.array(img)
enhanced = niclahe(img_np, tile_size=16, base_clip_limit=2.5, alpha=0.8)
```

What happens:

- The input medical image (e.g., a retinal photo) is loaded and converted to **grayscale** ("L" mode). Grayscale means each pixel has a single brightness value between 0 (black) and 255 (white).
- The image is turned into a NumPy array, a table of numbers the computer can work with.
- We then call the niclahe() function, giving it three key numbers:
    - tile_size: the side length of the small squares the image will be divided into (here 16×16 pixels).
    - base_clip_limit: a starting value for the "brake" that prevents over-enhancement; the code will later modify this per tile.
    - alpha: a sensitivity dial – larger alpha makes the clip limit more responsive to local contrast differences.

3. Padding the Image

python

```
pad_h = (tile_size - h % tile_size) % tile_size

pad_w = (tile_size - w % tile_size) % tile_size

img_padded = np.pad(img, ((0, pad_h), (0, pad_w)), mode='reflect')
```

Why needed:

We want to cut the image into an exact number of whole tiles. If the image width/height is not a perfect multiple of tile_size, we add a few extra rows/columns of pixels at the bottom and right edges. The reflect mode copies the edge pixels like a mirror, avoiding sharp artificial edges. After processing, we'll crop the image back to its original size.

4. Dividing the Image into Tiles

python

```
tiles_y = padded_h // tile_size
tiles_x = padded_w // tile_size
```

Here, tiles_y and tiles_x count how many tiles fit in the vertical and horizontal directions. This creates a grid of small rectangles – the neighborhood processing will later look at each of these tile positions.

5. Computing the Local Contrast of Each Tile (Tile Standard Deviation)

python

```
tile_stds = np.zeros((tiles_y, tiles_x), dtype=np.float32)
for ty in range(tiles_y):
    for tx in range(tiles_x):
        y0 = ty * tile_size
        y1 = y0 + tile_size
        x0 = tx * tile_size
        x1 = x0 + tile_size
        tile = img_padded[y0:y1, x0:x1]
        tile_stds[ty, tx] = np.std(tile)
```

What is standard deviation?

Think of it as a **measure of spread** – how much the brightness values inside the tile differ from the average.

- A **low standard deviation** means the tile is almost uniform (e.g., the dark background of the retina).
- A **high standard deviation** means the tile contains both bright and dark pixels (e.g., an area full of blood vessels and background).

The code calculates this "spread" for every tile and saves the numbers in a 2D table tile_stds. This is the raw measurement of local contrast.

6. Neighborhood Averaging – The Core of NICLAHE

python

```
padded_std = np.pad(tile_stds, ((1,1), (1,1)), mode='reflect')
neighbourhood_std = np.zeros_like(tile_stds)
for ty in range(tiles_y):
    for tx in range(tiles_x):
        patch = padded_std[ty:ty+3, tx:tx+3]
        neighbourhood_std[ty, tx] = np.mean(patch)
```

What's happening here?

For each tile, the algorithm **looks at its neighbors**. It takes a 3×3 block of tiles (the tile itself plus the eight tiles that surround it) and computes the **average of their standard deviations**. This average, stored in neighbourhood_std, represents the **typical contrast level in that region** – a smooth, reliable estimate that prevents decisions based on one isolated tile that might contain noise.

Why is this "neighborhood-based"?

This is the key improvement from the article. Standard CLAHE treats every tile independently; NICLAHE makes the enhancement decision (the clip limit) depend on the **local context**. A tile in a dark, flat area will have a low neighborhood average, while a tile near a detailed optic disc will have a high neighborhood average. The enhancement is then tailored accordingly.

7. Global Normalization – Making the Values Comparable

python

```
global_mean_std = np.mean(neighbourhood_std)
global_std_std = np.std(neighbourhood_std) + 1e-6
```

Here we compute the **overall average** (global_mean_std) and **overall spread** (global_std_std) of the neighborhood averages across the entire image. These numbers give a reference point: we can now express how far each tile's neighborhood contrast is from the image's average behavior. Adding 1e-6 avoids division by zero in case all tiles are flat.

8. Adaptive Clip Limit – Deciding How Much Enhancement Each Tile Gets

python

```
norm_std = (neighbourhood_std[ty, tx] - global_mean_std) / global_std_std

factor = 1.0 + alpha * (1.0 - 2.0 / (1.0 + np.exp(-norm_std)))

clip_limit = base_clip_limit * factor
```

Breaking it down:

- norm_std is a **z-score** – it tells us: "How many 'steps' above or below the average is the contrast of this tile's neighbourhood?" Positive = higher than average contrast (busy area).

  Negative = lower than average contrast (flat area).

- The expression (1.0 - 2.0 / (1.0 + np.exp(-norm_std))) is a smooth, S-shaped (sigmoid) function. It converts the z-score into a **factor** that gently swings around 1.0.
  - If norm_std is very **negative** (flat area), the factor becomes **larger than 1.0**.
  - If norm_std is very **positive** (textured area), the factor becomes **smaller than 1.0**.
  - The parameter alpha controls how strongly the factor responds; larger alpha → bigger swings.
- The final clip_limit for that tile is base_clip_limit * factor. So a **flat** tile gets a **higher clip limit**, allowing more contrast enhancement to bring out hidden details.

  A **textured** tile gets a **lower clip limit**, preventing over-amplification of noise and preserving natural appearance.

This is the **adaptive part** that distinguishes NICLAHE from regular CLAHE. It's like a smart camera that automatically brightens dark rooms more than sunny outdoors.

9. Tile-by-Tile Histogram Processing

Now the code loops over every tile again and performs **contrast-limited histogram equalization** with the tile-specific clip_limit.

a. Building the histogram

python

```
hist, _ = np.histogram(tile, bins=256, range=(0, 255))
hist = hist.astype(np.float32)
```

Counts how many pixels have each brightness level (0–255) inside the tile. This is just a bar chart of the tile's brightness distribution.

b. Clipping the histogram

python

```
total_excess = 0
for k in range(256):
    if hist[k] > clip_limit:
        total_excess += hist[k] - clip_limit
        hist[k] = clip_limit
```

If any brightness level appears too often (more than clip_limit), it is **chopped down** to the limit. The chopped-off amounts are added to a total_excess bucket. This prevents a single dominating brightness from being stretched too far and amplifying noise.

c. Redistributing the excess

python

```
if total_excess > 0:
    add = total_excess / 256.0
```

```
        hist += add
```

The excess that was cut off is spread **equally** over all 256 brightness levels. This keeps the total number of pixels unchanged while making the histogram more balanced – a core part of CLAHE.

d. Computing the mapping

python

```
        cdf = np.cumsum(hist)
        cdf_min = cdf[cdf > 0][0] if np.any(cdf > 0) else 0
        total_pixels = tile_size * tile_size
        if total_pixels != cdf_min:
            mapping = np.clip((cdf - cdf_min) / (total_pixels - cdf_min) * 255, 0, 255)
        else:
            mapping = np.arange(256, dtype=np.float32)
```

The cumulative distribution function (CDF) is built from the clipped histogram. Then a new brightness value is calculated for every original brightness level, such that the resulting histogram becomes more spread out (equalized). This mapping is stored for the tile.

The result for the whole grid is a 2D list of mappings (mappings[ty][tx]), each containing 256 numbers telling us what the new pixel brightness should be for every old brightness.

10. Bilinear Interpolation – Smooth Blending of Tiles

python

```
        for i in range(padded_h):
            for j in range(padded_w):
                # find four nearest tile centers
                ...
                new_val = (wy * wx * v00 + wy * (1 - wx) * v01 +
```

```
                (1 - wy) * wx * v10 + (1 - wy) * (1 - wx) * v11)
            result[i, j] = new_val
```

Instead of simply applying the mapping from the tile that contains the pixel, the code blends the mappings from the **four nearest tile centers**.

- For each pixel, it calculates fractional distances (wx, wy) to those centers.
- It retrieves the enhanced brightness from each of those four tiles' mappings (even doing a small interpolation *inside* the mapping if the brightness falls between two integers).
- The four values are then mixed together according to the distances – closer tiles influence the result more.

This is called **bilinear interpolation** and it completely eliminates the blocky edges that would appear if we used one mapping per tile. The output image looks smooth and natural.

11. Cropping and Saving

python

```
    result = result[:h, :w]
    return result.astype(np.uint8)
    ...
    Image.fromarray(enhanced).save(output_path)
```

Finally, the padded extra rows/columns are removed, the image is converted back to the standard 0-255 integer format, and saved. The result is a **contrast-enhanced medical image** where delicate details (like retinal blood vessels) are clearly visible, while noise and artefacts are kept under control.

12. Connection to the Article (https://doi.org/10.18240/ijo.2025.12.02)

The code directly mirrors the main contributions described in the paper:

1. **Neighborhood-based adaptation** – The clip limit is not fixed; it depends on the **average contrast of the surrounding tiles**. This is exactly the "neighborhood based" concept in NICLAHE.
2. **Adaptive clip limit per tile** – Flat, low-contrast regions get a higher clip limit (stronger enhancement), while textured regions get a lower clip limit (milder enhancement). This is the mathematical implementation of the idea that enhancement should be content-sensitive.
3. **Retinal image application** – The method is designed for medical images like retinal fundus photographs, where background and vessel regions require very different treatments. The adaptive behavior prevents the common problems of standard CLAHE (noise amplification in dark uniform areas, loss of detail in bright regions).

In the full paper, the authors also explore **adaptive tile size** selection; our implementation keeps tile size fixed but fully captures the spirit of the neighborhood-adaptive clip limit, which is the heart of their improvement.

Summary Table

Code Segment	What it does (in simple terms)	Link to NICLAHE paper
Padding	Extends the image so it can be evenly divided into square tiles.	Preprocessing to enable tile grid.
Tile standard deviation	Measures the "busyness" of each tile.	Measures local contrast.
Neighborhood averaging	Looks at a 3×3 block of tiles to get a smooth, region-wise contrast.	The **neighborhood** concept – avoids isolated noise.
Global normalization	Finds the average contrast of the whole image.	Provides a reference for adaptation.
Sigmoid factor & adaptive clip limit	Converts local contrast into a multiplier; flat areas get a higher limit, busy areas lower.	The **adaptive clip limit** – the key novel contribution.

Histogram clipping & redistribution	Prevents any one brightness from dominating, then re-spreads the excess.	Standard CLAHE contrast limiting.
CDF & mapping	Stretches brightness values to cover the full range.	Histogram equalization core.
Bilinear interpolation	Smoothly blends tile enhancements so no block boundaries are visible.	Avoids tile-border artefacts.